

SQL Project

1. Creation and insertion of data into Customers table

The screenshot shows the SQL Developer interface with the 'sqlproject' schema selected in the Navigator. The SQL Editor contains the following SQL statements:

```
35 • INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (24, 'Brandy Wright', 'user24@example.com', '142-479-1945', '2024-10-14 16:01:12');
36 • INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (25, 'Cindy Hart', 'user25@example.com', '9153947511', '2023-11-25 15:45:24');
37 • INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (26, 'Victoria Hall', 'user26@example.com', '2073473421', '2024-06-23 09:55:22');
38 • INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (27, 'Adrienne Green', 'user27@example.com', '586-605-5061x06', '2024-10-27 17:19:38');
39 • INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (28, 'Joseph Stuart', 'user28@example.com', '(751)456-8289x1', '2023-10-29 02:43:00');
40 • INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (29, 'Kara Zavala', 'user29@example.com', '7074-06-74-03-77-50', '2024-06-24 03:77:50');
41 • INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (30, 'Victoria Acevedo', 'user30@example.com', '7074-06-74-03-77-50', '2024-06-24 03:77:50');
42
43 • select * from customers;
```

The Result Grid shows the data inserted into the 'customers' table:

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
6	Mark Duke	user6@example.com	7074-06-74-03-77-50	2024-06-24 03:77:50

The Output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
35	23:56:49	INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (...)	1 row(s) affected	0.000 sec
36	23:57:07	select * from customers LIMIT 0, 500	30 row(s) returned	0.000 sec / 0.000 sec

2. Creation and insertion of data in products table

The screenshot shows the SQL Developer interface with the 'sqlproject' schema selected in the Navigator. The SQL Editor contains the following SQL statements:

```
93 • INSERT INTO products (product_id, name, category, price, stock_quantity, added_on) VALUES (44, 'Actually T', 'Clothing', 639.43, 152, '2024-01-30 06:30:53');
94 • INSERT INTO products (product_id, name, category, price, stock_quantity, added_on) VALUES (45, 'Suffer We', 'Electronics', 4813.68, 84, '2025-05-30 10:02:50');
95 • INSERT INTO products (product_id, name, category, price, stock_quantity, added_on) VALUES (46, 'Age Treatm', 'Electronics', 2529.51, 101, '2025-04-13 01:11:46');
96 • INSERT INTO products (product_id, name, category, price, stock_quantity, added_on) VALUES (47, 'Southern T', 'Clothing', 4759.28, 179, '2025-06-03 13:34:03');
97 • INSERT INTO products (product_id, name, category, price, stock_quantity, added_on) VALUES (48, 'Group But', 'Clothing', 4722.66, 75, '2023-11-06 00:47:37');
98 • INSERT INTO products (product_id, name, category, price, stock_quantity, added_on) VALUES (49, 'Rest Somet', 'Electronics', 177.40, 64, '2024-12-10 10:37:14');
99 • INSERT INTO products (product_id, name, category, price, stock_quantity, added_on) VALUES (50, 'Stock Arti', 'Electronics', 177.40, 64, '2024-12-10 10:37:14');
100
101 • Select * from products;
```

The Result Grid shows the data inserted into the 'products' table:

product_id	name	category	price	stock_quantity	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
6	16 Whisker	Electronics	177.40	64	2024-12-10 10:37:14

The Output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
87	00:06:42	INSERT INTO products (product_id, name, category, price, stock_quantity, added_...	1 row(s) affected	0.000 sec
88	00:07:10	Select * from products LIMIT 0, 500	50 row(s) returned	0.000 sec / 0.000 sec

3. Creation and insertion of data in orders table

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'SCHEMAS' tree with the 'sqlproject' database selected. The right pane shows the 'SQL File 4*' editor with the following SQL script:

```
503 • INSERT INTO orders (order_id, customer_id, order_date, status, total_amount) VALUES (395, 9, '2025-02-11 0
504 • INSERT INTO orders (order_id, customer_id, order_date, status, total_amount) VALUES (396, 28, '2024-07-21
505 • INSERT INTO orders (order_id, customer_id, order_date, status, total_amount) VALUES (397, 10, '2024-11-10
506 • INSERT INTO orders (order_id, customer_id, order_date, status, total_amount) VALUES (398, 17, '2024-11-07
507 • INSERT INTO orders (order_id, customer_id, order_date, status, total_amount) VALUES (399, 17, '2025-02-08
508 • INSERT INTO orders (order_id, customer_id, order_date, status, total_amount) VALUES (400, 7, '2024-10-22 1
509
510
511 • select * from orders;
```

The 'Result Grid' shows the data inserted into the 'orders' table:

order_id	customer_id	order_date	status	total_amount
1	20	2025-03-02 07:20:11	Delivered	9414.28
2	18	2024-10-09 18:08:21	Shipped	532.20
3	15	2025-05-08 00:08:27	Cancelled	5164.56
4	11	2024-09-19 22:16:13	Delivered	9469.78
5	12	2025-04-08 18:02:06	Pending	14501.86

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
✓ 489	00:13:03	INSERT INTO orders (order_id, customer_id, order_date, status, total_amount) VAL...	1 row(s) affected	0.000 sec
✓ 490	00:13:18	select * from orders LIMIT 0, 500	400 row(s) returned	0.000 sec / 0.000 sec

A tooltip on the right states: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

4. Creation and insertion of data into order_item table

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'SCHEMAS' tree with the 'sqlproject' database selected. The right pane shows the 'SQL File 4*' editor with the following SQL script:

```
1723 • INSERT INTO order_items (order_item_id, order_id, product_id, quantity, item_price) VALUES (1196, 399, 31,
1724 • INSERT INTO order_items (order_item_id, order_id, product_id, quantity, item_price) VALUES (1197, 399, 16,
1725 • INSERT INTO order_items (order_item_id, order_id, product_id, quantity, item_price) VALUES (1198, 400, 47,
1726 • INSERT INTO order_items (order_item_id, order_id, product_id, quantity, item_price) VALUES (1199, 400, 1,
1727 • INSERT INTO order_items (order_item_id, order_id, product_id, quantity, item_price) VALUES (1200, 400, 43,
1728 • INSERT INTO order_items (order_item_id, order_id, product_id, quantity, item_price) VALUES (1201, 400, 14,
1729
1730
1731 • select * from order_items;
```

The 'Result Grid' shows the data inserted into the 'order_items' table:

order_item_id	order_id	product_id	quantity	item_price
1	1	19	2	4707.14
2	2	6	3	177.40
3	3	9	3	985.19
4	3	23	1	2208.99
5	4	35	2	4734.89

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
✓ 1692	00:21:22	INSERT INTO order_items (order_item_id, order_id, product_id, quantity, item_price)...	1 row(s) affected	0.000 sec
✓ 1693	00:21:39	select * from order_items LIMIT 0, 500	500 row(s) returned	0.000 sec / 0.000 sec

A tooltip on the right states: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

5. Creation and insertion of data in payments table

The screenshot shows the SQL Developer interface with the 'payments' table selected in the Schemas pane. The SQL Editor displays a series of INSERT statements for the 'payments' table, followed by a SELECT statement to verify the data. The Result Grid shows the data inserted, and the Output pane shows the execution results.

SQL Editor:

```
2145 • INSERT INTO payments (payment_id, order_id, payment_date, amount_paid, method) VALUES (394, 394, '2025-06-
2146 • INSERT INTO payments (payment_id, order_id, payment_date, amount_paid, method) VALUES (395, 395, '2025-02-
2147 • INSERT INTO payments (payment_id, order_id, payment_date, amount_paid, method) VALUES (396, 396, '2024-07-
2148 • INSERT INTO payments (payment_id, order_id, payment_date, amount_paid, method) VALUES (397, 397, '2024-11-
2149 • INSERT INTO payments (payment_id, order_id, payment_date, amount_paid, method) VALUES (398, 398, '2024-11-
2150 • INSERT INTO payments (payment_id, order_id, payment_date, amount_paid, method) VALUES (399, 399, '2025-02-
2151 • INSERT INTO payments (payment_id, order_id, payment_date, amount_paid, method) VALUES (400, 400, '2024-10-
2152
2153 • select * from payments;
```

Result Grid:

payment_id	order_id	payment_date	amount_paid	method
1	1	2025-03-02 08:16:11	9414.28	Credit Card
2	2	2024-10-09 19:00:21	532.20	Net Banking
3	3	2025-05-08 00:55:27	5164.56	Credit Card
4	4	2024-09-19 22:28:13	9469.78	UPI
5	5	2025-04-08 18:26:06	14501.86	UPI
6	6	2024-10-25 08:16:09	21050.17	UPI

Output:

#	Time	Action	Message	Duration / Fetch
✓	2095 00:31:06	INSERT INTO payments (payment_id, order_id, payment_date, amount_paid, meth...	1 row(s) affected	0.000 sec
✓	2096 00:31:28	select * from payments LIMIT 0, 500	400 row(s) returned	0.000 sec / 0.000 sec

6. Creation and insertion of data in product_reviews table

The screenshot shows the SQL Developer interface with the 'product_reviews' table selected in the Schemas pane. The SQL Editor displays a series of INSERT statements for the 'product_reviews' table, followed by a SELECT statement to verify the data. The Result Grid shows the data inserted, and the Output pane shows the execution results.

SQL Editor:

```
2219 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES
2220 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES
2221 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES
2222 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES
2223 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES
2224 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES
2225 • INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES
2226
2227 • Select * from product_reviews;
```

Result Grid:

review_id	product_id	customer_id	rating	review_text	review_date
1	1	21	2	Great product, loved it!	2024-11-07 02:40:51
2	9	27	4	Product arrived damaged, had to return.	2025-06-08 02:40:51
3	42	21	5	Very fast delivery, happy with the purchase.	2024-10-01 02:40:51
4	13	28	2	Average quality, but acceptable.	2024-12-23 02:40:51
5	45	9	4	Good value for money.	2024-10-07 02:40:51
6	47	30	5	Product arrived damaged, had to return.	2024-07-06 02:40:51

Output:

#	Time	Action	Message	Duration / Fetch
✓	2148 00:39:45	INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review...	1 row(s) affected	0.015 sec
✓	2149 00:40:16	Select * from product_reviews LIMIT 0, 500	50 row(s) returned	0.000 sec / 0.000 sec

Questions

Level 1: Basics

1. Retrieve Customer name and email for email marketing

Select name,email from customers;

```
3 • select name,email from customers;
```

	name	email
▶	Thomas Owens	user1@example.com
	Charles Grant	user2@example.com
	Kaitlin Richards	user3@example.com
	Christina Williams	user4@example.com
	David Allen	user2@example.com
	Mark Duke	user6@example.com

customers 2 x

Read Only

2. View complete product catalog with all available details

select * from products;

```
3 • select * from products;
```

	product_id	name	category	price	stock_quantity	added_on
▶	1	Plant No	Home	639.43	152	2024-01-30 06:30:53
	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14

products 1 x

Apply Revert

3. List all unique product category

`select distinct(category) from products;`

5 • `select distinct(category) from products;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

category
Home
Clothing
Electronics
Toys
Books

Result 2 x Read Only

4. Show all products priced above Rs. 1000

`select * from products where price > 1000 order by price asc;`

7 • `select * from products where price > 1000 order by price asc;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

product_id	name	category	price	stock_quantity	added_on
27	Answer But	Home	1016.68	90	2025-04-06 21:54:11
16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
43	Least Green	Toys	1398.26	81	2024-04-10 15:23:33
37	Between Up	Toys	1429.45	61	2025-01-27 01:30:37
11	Build Her	Books	1852.64	150	2024-08-08 01:00:15

products 5 x Apply Revert

5. Display products within a mid range price bracket(Rs. 2000 to Rs. 5000)

```
select *  
from products  
where price between 2000 and 5000  
order by price asc;
```

```
9 • select *  
10 from products  
11 where price between 2000 and 5000  
12 order by price asc;
```

product_id	name	category	price	stock_quantity	added_on
31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
23	Life Series	Clothing	2208.99	136	2024-03-09 09:19:13
32	Factor Where	Factor Where	2295.14	167	2023-07-07 00:59:39
28	East Foot	toys	2414.93	128	2025-05-11 16:02:29
17	Everything Plant	Books	2406.68	170	2023-10-08 20:11:55

products 7 x

Apply Revert

6. Fetch data for specific customer IDs(e.g from loyalty program list)

```
select *  
from customers  
where customer_id in (30,29,25,12);
```

```
14 • select *  
15 from customers  
16 where customer_id in (30,29,25,12);
```

customer_id	name	email	phone	created_at
12	Mary Knight	user12@example.com	361-636-3802	2023-08-16 20:05:50
25	Cindy Hart	user25@example.com	617.574.8421x41	2025-05-27 18:02:54
29	Kara Zavala	user29@example.com	434-985-5776x61	2023-09-27 15:57:09
30	Victoria Acevedo	user30@example.com	+1-325-063-7244	2023-11-28 12:30:59
*	NULL	NULL	NULL	NULL

customers 13 x

Apply Revert

7. Identify customers whose names start with the letter 'A'

```
select *  
from customers  
where name like 'A%';
```

```
14 • select *  
15 from customers  
16 where name like 'A%';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	customer_id	name	email	phone	created_at
▶	15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
	16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
	19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
	27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29
*	NULL	NULL	NULL	NULL	NULL

customers 14 x Apply Revert

8. List electronics products priced under ₹3,000

```
Select *  
from products  
where category = 'Electronics' and price < 3000  
order by price desc;
```

```
18 • Select *  
19 from products  
20 where category = 'Electronics' and price < 3000  
21 order by price desc;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	product_id	name	category	price	stock_quantity	added_on
▶	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
	34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
	15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
	47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38
	44	Actually Term	Electronics	306.11	85	2023-11-02 13:08:30

products 17 x Apply Revert

9. Display product names and prices in descending order of price

Select name,price
from products
order by price desc;

The screenshot shows a database query interface. At the top, a SQL query is entered in a text area:

```
23 • Select name, price
24 from products
25 order by price desc;
```

Below the query, there is a toolbar with options: "Result Grid" (selected), "Filter Rows:", "Export:", and "Wrap Cell Content:". The main area displays a table of results:

	name	price
▶	Response Indeed	4897.36
	Population Social	4813.68
	Development System	4801.78
	Any Question	4759.28
	Fire Often	4734.89
	Natural Network	4722.66

At the bottom left, there is a tab labeled "products 19" with a close button. At the bottom right, there is a "Read Only" status indicator.

10. Display product names and prices, sorted by price and then by name

Select name,price
from products
order by price desc, name asc;

The screenshot shows a database query interface. At the top, a SQL query is entered in a text area:

```
23 • Select name,price
24 from products
25 order by price desc, name asc;
```

Below the query, there is a toolbar with options: "Result Grid" (selected), "Filter Rows:", "Export:", and "Wrap Cell Content:". The main area displays a table of results:

	name	price
▶	Response Indeed	4897.36
	Population Social	4813.68
	Development System	4801.78
	Any Question	4759.28
	Fire Often	4734.89
	Natural Network	4722.66

At the bottom left, there is a tab labeled "products 30" with a close button. At the bottom right, there is a "Read Only" status indicator.

Level 2: Filtering and Formatting

1. Retrieve orders where customer information is missing (possibly due to data migration or deletion).

```
select *  
from orders  
where customer_id not in ( Select customer_id  
                           from customers  
                           where customer_id is not null);
```

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
27 • select *  
28   from orders  
29   where customer_id not in ( Select customer_id  
30                             from customers  
31                             where customer_id is not null)
```

Below the query editor is the 'Result Grid' tab. It shows a table with 5 columns: order_id, customer_id, order_date, status, and total_amount. The first row of data shows all five columns as NULL. The interface includes a toolbar with options like 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'. On the right side, there are buttons for 'Result Grid' and 'Form Editor'. At the bottom, a status bar indicates 'orders 38' and has 'Apply' and 'Revert' buttons.

order_id	customer_id	order_date	status	total_amount
NULL	NULL	NULL	NULL	NULL

2. Display customer names and emails using column aliases for frontend readability

```
select name as 'Customer Name', email as EmailId  
from customers;
```

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
33 • select name as 'Customer Name', email as EmailId  
34   from customers;
```

Below the query editor is the 'Result Grid' tab. It shows a table with 2 columns: Customer Name and EmailId. The first row of data shows Thomas Owens and user1@example.com. The interface includes a toolbar with options like 'Filter Rows', 'Export', and 'Wrap Cell Content'. On the right side, there are buttons for 'Result Grid' and 'Form Editor'. At the bottom, a status bar indicates 'customers 41' and has a 'Read Only' indicator.

Customer Name	EmailId
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com

3. Calculate total value per item ordered by multiplying quantity and item price

```
select *, quantity*item_price as 'Grand Total'
from order_items;
```

```
36 • select *, quantity*item_price as 'Grand Total'
37   from order_items;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:	Result Grid	Form Editor
order_item_id	order_id	product_id	quantity	item_price	Grand Total	
1	1	19	2	4707.14	9414.28	
2	2	6	3	177.40	532.20	
3	3	9	3	985.19	2955.57	
4	3	23	1	2208.99	2208.99	
5	4	35	2	4734.89	9469.78	

Result 47 × Read Only

4. Combine customer name and phone number in a single column

```
select concat(name, phone) as NameAndPhone
from customers;
```

```
39 • select concat(name, phone) as NameAndPhone
40   from customers;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Result Grid	Form Editor
NameAndPhone					
Thomas Owens142-479-1945					
Charles Grant9153947511					
Kaitlin Richards2073473421					
Christina Williams586-605-5061x06					
David Allen(751)456-8289x1					
Mark Duhaime(144)057-7811					

Result 50 × Read Only

5. Extract only the date part from order timestamps for date-wise reporting

Select `date(order_date)` from `orders`;

The screenshot shows a SQL query editor with the query `select date(order_date) from orders;` entered. Below the query, the results are displayed in a grid. The grid has one column labeled `date(order_date)` and contains several rows of dates. The interface includes a toolbar with options like 'Filter Rows', 'Export', and 'Wrap Cell Content'. On the right side, there are buttons for 'Result Grid' and 'Form Editor'. At the bottom, there is a 'Read Only' indicator.

date(order_date)
2025-03-02
2024-10-09
2025-05-08
2024-09-19
2025-04-08
2024-10-25

6. List products that do not have any stock left

select *
from products
where stock_quantity is Null
or stock_quantity = 0;

The screenshot shows a SQL query editor with the query `select * from products where stock_quantity is Null or stock_quantity = 0;` entered. Below the query, the results are displayed in a grid. The grid has six columns: `product_id`, `name`, `category`, `price`, `stock_quantity`, and `added_on`. The first row shows all values as `NULL`. The interface includes a toolbar with options like 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'. On the right side, there are buttons for 'Result Grid' and 'Form Editor'. At the bottom, there are 'Apply' and 'Revert' buttons.

product_id	name	category	price	stock_quantity	added_on
*	NULL	NULL	NULL	NULL	NULL

Level 3: Aggregations

1. Count the total number of orders placed

Select count(*) as TotalOrders
from orders;

```
49 • Select count(*) as TotalOrders
50 from orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

TotalOrders
400

Result 2 x Read Only

2. Calculate the total revenue collected from all orders

select sum(total_amount) as TotalRevenue
from orders;

```
52 • select sum(total_amount) as TotalRevenue
53 from orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

TotalRevenue
6960973.66

Result 3 x Read Only

3. Calculate the average order value

Select avg(total_amount) as AverageRevenue
from orders;

```
55 • Select avg(total_amount) as AverageRevenue
56 from orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

AverageRevenue
17402.434150

Result 4 x Read Only

4. Count the number of customers who have placed at least one order

Select count(distinct(customer_id)) as active_customers
from orders;

```
58 • Select count(distinct(customer_id)) as active_customers
59 from orders;
```

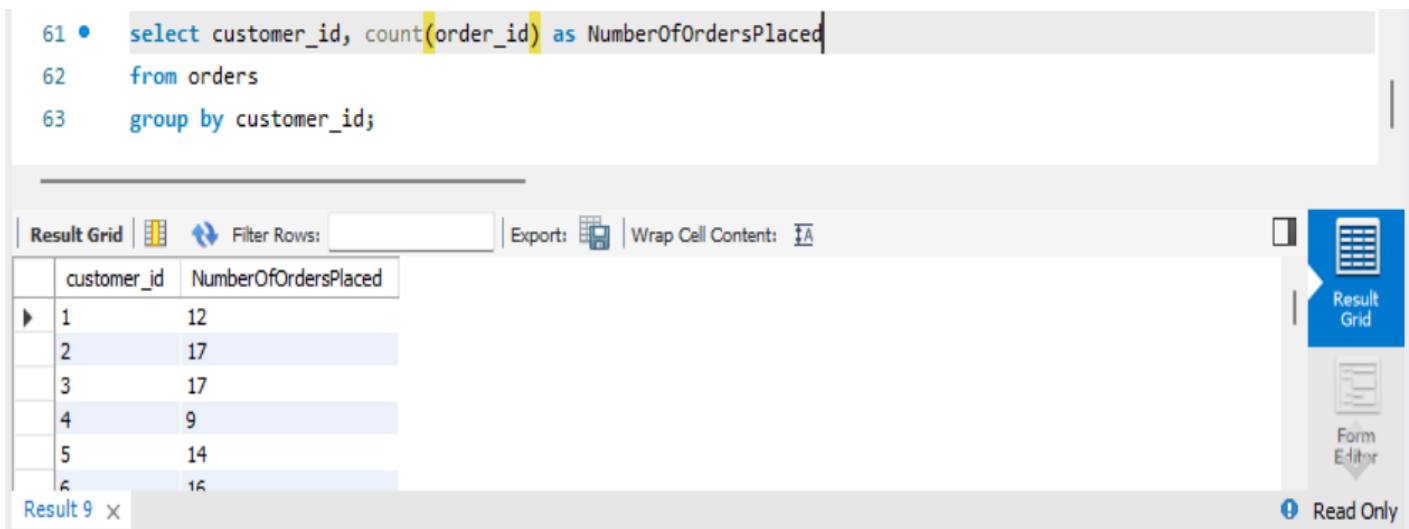
Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

active_customers
30

Result 6 x Read Only

5. Find the number of orders placed by each customer

```
select customer_id, count(order_id) as NumberOfOrdersPlaced
from orders
group by customer_id;
```



61 • select customer_id, count(order_id) as NumberOfOrdersPlaced

62 from orders

63 group by customer_id;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

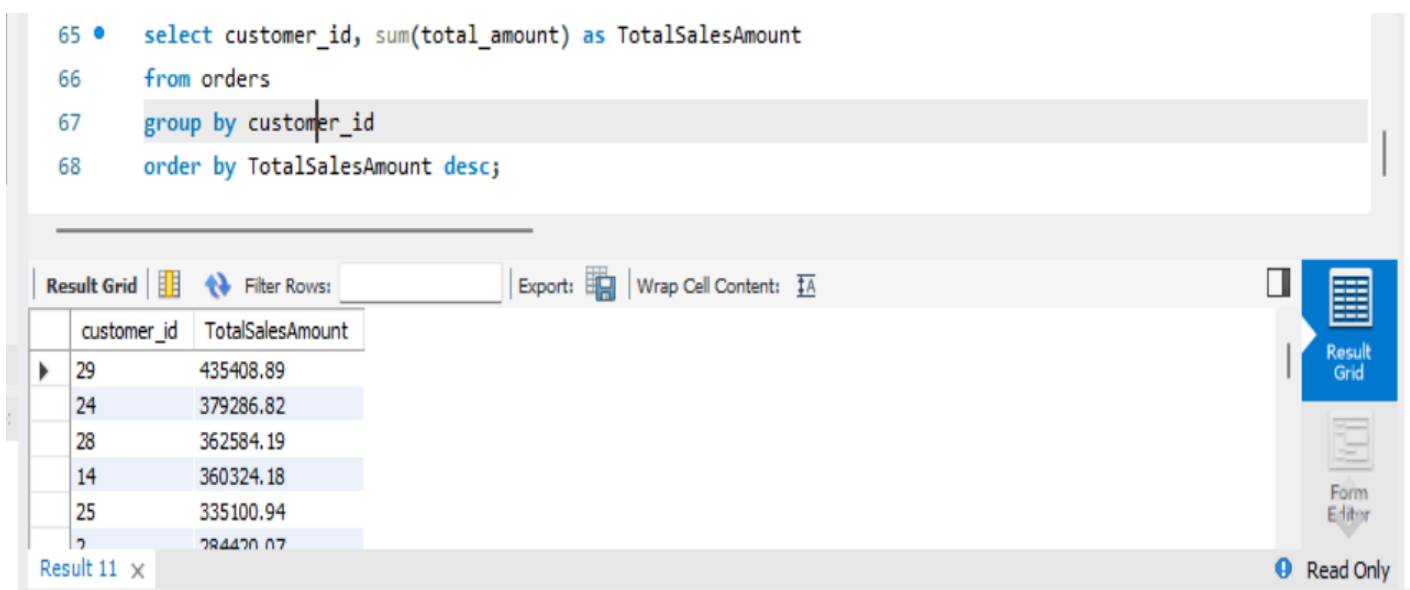
	customer_id	NumberOfOrdersPlaced
▶	1	12
	2	17
	3	17
	4	9
	5	14
	6	16

Result 9 x

Read Only

6. Find total sales amount made by each customer

```
select customer_id, sum(total_amount) as TotalSalesAmount
from orders
group by customer_id
order by TotalSalesAmount desc;
```



65 • select customer_id, sum(total_amount) as TotalSalesAmount

66 from orders

67 group by customer_id

68 order by TotalSalesAmount desc;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_id	TotalSalesAmount
▶	29	435408.89
	24	379286.82
	28	362584.19
	14	360324.18
	25	335100.94
	7	284470.07

Result 11 x

Read Only

7. List the number of products sold per category

```
select p.category as Category, sum(oi.quantity) as ProductsFromCategorySold
from products as p inner join order_items as oi
on p.product_id = oi.product_id
group by Category;
```

```
70 • select p.category as Category, sum(oi.quantity) as ProductsFromCategorySold
71 from products as p inner join order_items as oi
72 on p.product_id = oi.product_id
73 group by Category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Category	ProductsFromCategorySold
▶	Home	443
	Clothing	559
	Electronics	687
	Toys	405
	Books	350

Result 13 x Read Only

8. Find the average item price per category

```
select category, round(avg(price),2) as AverageItemPrice
from products
group by category;
```

```
75 • select category, round(avg(price),2) as AverageItemPrice
76 from products
77 group by category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	category	AverageItemPrice
▶	Home	2146.37
	Clothing	3434.58
	Electronics	2653.39
	Toys	2516.53
	Books	3167.08

Result 15 x Read Only

9. Show number of orders placed per day

```
select date(order_date) as OrderDate, count(order_id)
from orders
group by OrderDate;
```

```
79 • select date(order_date) as OrderDate, count(order_id)
80   from orders
81   group by OrderDate;
```

OrderDate	count(order_id)
2025-03-02	3
2024-10-09	2
2025-05-08	2
2024-09-19	3
2025-04-08	3
2024-10-25	3

Result 16 x

Read Only

10. List total payments received per payment method

```
Select method, sum(amount_paid) as TotalPaymentReceived
from payments
group by method;
```

```
83 • Select method, sum(amount_paid) as TotalPaymentReceived
84   from payments
85   group by method;
```

method	TotalPaymentReceived
Credit Card	1754603.10
Net Banking	1658383.90
UPI	1617408.78
Debit Card	1930577.88

Result 18 x

Read Only

Level 4: Multi-Table Queries (JOINS)

1. Retrieve order details along with the customer name (INNER JOIN)

```
select c.name as CustomerName, o.*
from customers c inner join orders o
on c.customer_id = o.customer_id;
```

```
87 • select c.name as CustomerName, o.*
88 from customers c inner join orders o
89 on c.customer_id = o.customer_id;
```

Result Grid							Filter Rows:		Export:	Wrap Cell Content:		
	CustomerName	order_id	customer_id	order_date	status	total_amount						
▶	Thomas Owens	14	1	2024-09-24 21:21:38	Shipped	15803.34						
	Thomas Owens	17	1	2024-08-19 21:17:57	Pending	11173.77						
	Thomas Owens	61	1	2024-12-26 23:21:58	Shipped	13053.00						
	Thomas Owens	76	1	2025-01-14 22:59:54	Pending	23506.81						
	Thomas Owens	92	1	2024-09-26 11:33:58	Shipped	834.75						
	Thomas Owens	100	1	2025-03-17 04:56:18	Shipped	12100.14						

Result 19 x

Read Only

2. Get list of products that have been sold (INNER JOIN with order_items)

```
select distinct(p.name) as ProductName
from products as p inner join order_items as oi
on p.product_id = oi.product_id;
```

```
91 • select distinct(p.name) as ProductName
92 from products as p inner join order_items as oi
93 on p.product_id = oi.product_id;
94
```

Result Grid							Filter Rows:		Export:	Wrap Cell Content:		
	ProductName											
	Population Social											
	Available Answer											
	Any Question											
	Natural Network											
	If Whatever											
	Response Indeed											
	Every Amount											

Result 22 x

Read Only

3. List all orders with their payment method (INNER JOIN)

```
Select o.*,p.method  
from orders o inner join payments p  
on o.order_id = p.order_id;
```

```
96 • Select o.*,p.method  
97 from orders o inner join payments p  
98 on o.order_id = p.order_id;
```

	order_id	customer_id	order_date	status	total_amount	method
▶	1	20	2025-03-02 07:20:11	Delivered	9414.28	Credit Card
	2	18	2024-10-09 18:08:21	Shipped	532.20	Net Banking
	3	15	2025-05-08 00:08:27	Cancelled	5164.56	Credit Card
	4	11	2024-09-19 22:16:13	Delivered	9469.78	UPI
	5	12	2025-04-08 18:02:06	Pending	14501.86	UPI
	6	29	2024-10-25 07:33:59	Cancelled	31050.17	UPI
	7	22	2024-07-29 11:58:47	Shipped	3043.67	UPI

Result 24 x

Read Only

4. Get list of customers and their orders (LEFT JOIN)

```
select c.*,o.*  
from customers c left join orders o  
on c.customer_id = o.customer_id;
```

```
100 • select c.*,o.*  
101 from customers c left join orders o  
102 on c.customer_id = o.customer_id;
```

	name	email	phone	created_at	order_id	customer_id	order_date	status	total_amount
▶	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	14	1	2024-09-24 21:21:38	Shipped	15803.34
	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	17	1	2024-08-19 21:17:57	Pending	11173.77
	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	61	1	2024-12-26 23:21:58	Shipped	13053.00
	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	76	1	2025-01-14 22:59:54	Pending	23506.81
	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	92	1	2024-09-26 11:33:58	Shipped	834.75
	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	109	1	2025-03-17 04:56:18	Shipped	12190.14

Result 25 x

Read Only

5. List all products along with order item quantity (LEFT JOIN)

```
select p.*, oi.quantity as quantity
from products as p left join order_items oi
on p.product_id = oi.product_id;
```

```
104 • select p.*, oi.quantity as quantity
105 from products as p left join order_items oi
106 on p.product_id = oi.product_id;
```

product_id	name	category	price	stock_quantity	added_on	quantity
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	1
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	3
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	1
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	1
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	2
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	3
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	2

Result 26 x Read Only

6. List all payments including those with no matching orders (RIGHT JOIN)

```
Select p.method, o.*
from orders o right join payments p
on o.order_id = p.order_id;
```

```
108 • Select p.method, o.*
109 from orders o right join payments p
110 on o.order_id = p.order_id;
```

method	order_id	customer_id	order_date	status	total_amount
Credit Card	1	20	2025-03-02 07:20:11	Delivered	9414.28
Net Banking	2	18	2024-10-09 18:08:21	Shipped	532.20
Credit Card	3	15	2025-05-08 00:08:27	Cancelled	5164.56
UPI	4	11	2024-09-19 22:16:13	Delivered	9469.78
UPI	5	12	2025-04-08 18:02:06	Pending	14501.86
UPI	6	29	2024-10-25 07:33:59	Cancelled	31050.17
UPI	7	22	2024-07-29 11:58:47	Shipped	3043.67

Result 28 x Read Only

7. Combine data from three tables: customer, order, and payment

```
SELECT c.customer_id, o.order_id, p.payment_id
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
INNER JOIN payments p ON o.order_id = p.order_id;
```

```
113 • SELECT c.customer_id, o.order_id, p.payment_id
114 FROM customers c
115 INNER JOIN orders o ON c.customer_id = o.customer_id
116 INNER JOIN payments p ON o.order_id = p.order_id;
117
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

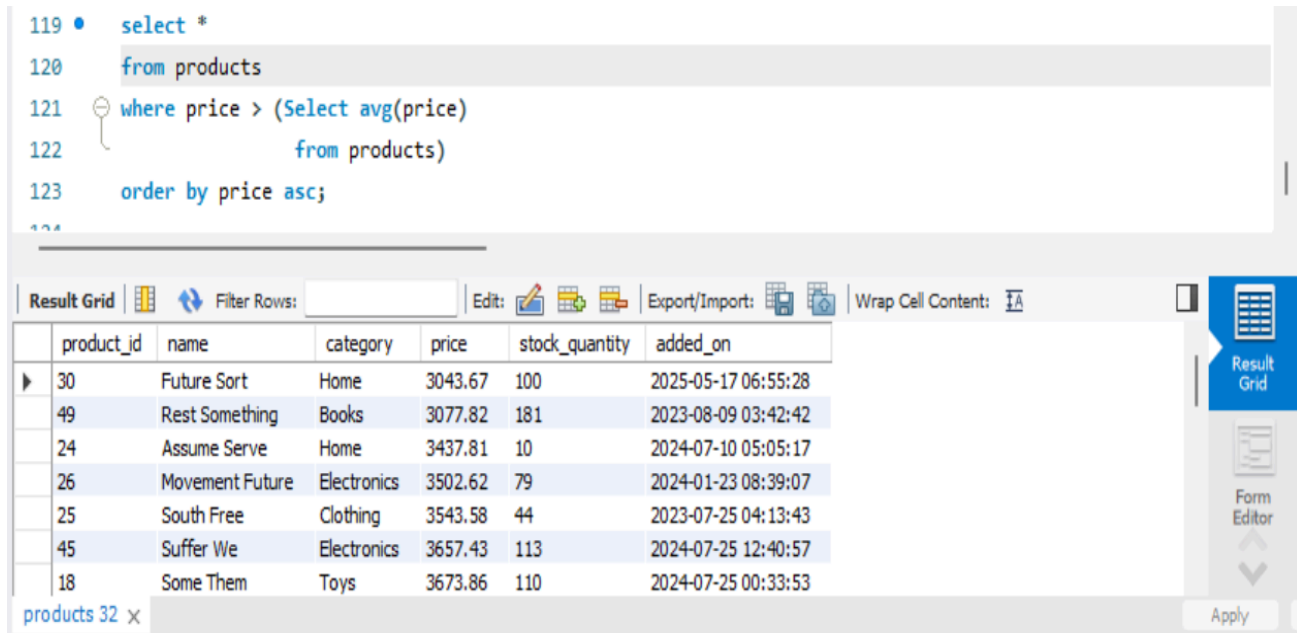
	customer_id	order_id	payment_id
▶	1	14	14
	1	17	17
	1	61	61
	1	76	76
	1	92	92
	1	109	109
	1	127	127

Result 30 x Read Only

Level 5: Subqueries (Inner Queries)

1. List all products priced above the average product price

```
select *  
from products  
where price > (Select avg(price)  
               from products)  
order by price asc;
```



The screenshot shows a database query editor with a SQL query and its results. The query is as follows:

```
119 • select *  
120   from products  
121   where price > (Select avg(price)  
122                  from products)  
123   order by price asc;
```

The results are displayed in a table with the following columns: product_id, name, category, price, stock_quantity, and added_on. The table contains 32 rows, with the first 7 rows visible in the screenshot.

product_id	name	category	price	stock_quantity	added_on
30	Future Sort	Home	3043.67	100	2025-05-17 06:55:28
49	Rest Something	Books	3077.82	181	2023-08-09 03:42:42
24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17
26	Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07
25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43
45	Suffer We	Electronics	3657.43	113	2024-07-25 12:40:57
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53

The interface includes a toolbar with options like Filter Rows, Edit, Export/Import, and Wrap Cell Content. A sidebar on the right contains buttons for Result Grid, Form Editor, and Apply.

2. Find customers who have placed at least one order

```
select *  
from customers  
where customer_id in (Select customer_id  
                      from orders  
                      where order_id is not null);
```

```

126 • select *
127   from customers
128  where customer_id in (Select customer_id
129                        from orders
130                        where order_id is not null);

```

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:	Result Grid	Form Editor
customer_id	name	email	phone	created_at		
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12		
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24		
3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22		
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38		
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00		
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59		
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43		

customers 33 x Apply

3. Show orders whose total amount is above the average for that customer

```

select order_id,customer_id,status,total_amount
from orders as o1
where total_amount>(Select avg(total_amount)
                    from orders as o2
                    where o1.customer_id = o2.customer_id)
order by customer_id asc;

```

```

133 • select order_id,customer_id,status,total_amount
134   from orders as o1
135  where total_amount>(Select avg(total_amount)
136                      from orders as o2
137                      where o1.customer_id = o2.customer_id)
138   order by customer_id asc;
139

```

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:	Result Grid	Form Editor
order_id	customer_id	status	total_amount			
14	1	Shipped	15803.34			
76	1	Pending	23506.81			
127	1	Pending	20160.64			
278	1	Delivered	32015.16			
379	1	Cancelled	21586.89			
223	2	Cancelled	41020.75			
230	2	Pending	22655.32			

orders 37 x Apply

4. Display customers who haven't placed any orders

```
Select *  
from customers  
where customer_id not in(select customer_id  
                        from orders  
                        where customer_id is not null);
```

```
142 • Select *  
143   from customers  
144   where customer_id not in(select customer_id  
145                           from orders  
146                           where customer_id is not null);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	customer_id	name	email	phone	created_at
*	NULL	NULL	NULL	NULL	NULL

customers 39 x Apply

5. Show products that were never ordered

```
Select *  
from products  
where product_id not in (select product_id  
                        from order_items  
                        where product_id is not null);
```

```
149 • Select *  
150   from products  
151   where product_id not in (select product_id  
152                           from order_items  
153                           where product_id is not null);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	product_id	name	category	price	stock_quantity	added_on
*	NULL	NULL	NULL	NULL	NULL	NULL

products 40 x Apply

6. Show highest value order per customer

```
Select customer_id,total_amount
from orders as o1
where total_amount = (Select max(total_amount)
                      from orders as o2
                      where o1.customer_id = o2.customer_id)
order by customer_id asc;
```

```
156 • Select customer_id,total_amount
157   from orders as o1
158   where total_amount = (Select max(total_amount)
159                         from orders as o2
160                         where o1.customer_id = o2.customer_id)
161   order by customer_id asc;
```

customer_id	total_amount
1	32015.16
2	42056.04
3	41679.11
4	25747.34
5	39921.78

orders 44 x

Read Only

7. Highest Order Per Customer (Including Names)

```
Select c.*, o.total_amount
from customers as c inner join orders as o
on c.customer_id = o.customer_id
where o.total_amount = (select max(total_amount)
                        from orders as o2
                        where o.customer_id = o2.customer_id);
```

```
164 • Select c.*, o.total_amount
165   from customers as c inner join orders as o
166   on c.customer_id = o.customer_id
167   where o.total_amount = (select max(total_amount)
168                           from orders as o2
169                           where o.customer_id = o2.customer_id);
```

customer_id	name	email	phone	created_at	total_amount
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	32015.16
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24	42056.04
3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22	41679.11
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38	25747.34
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00	39921.78
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59	39003.19

Result 48 x

Read Only

Level 6: Set Operations

1. List all customers who have either placed an order or written a product review

```
select customer_id,name
from customers
where customer_id in (Select customer_id
                      from orders)
or customer_id in (Select customer_id
                  from product_reviews
                  where review_text is not null);
```

The screenshot shows a database query editor with a SQL query and its results. The query is as follows:

```
172 • select customer_id,name
173   from customers
174   where customer_id in (Select customer_id
175                        from orders)
176   or customer_id in (Select customer_id
177                    from product_reviews
178                    where review_text is not null);
```

Below the query editor, there is a toolbar with options like "Result Grid", "Filter Rows", "Edit", "Export/Import", and "Wrap Cell Content". The "Result Grid" is active, displaying the following data:

customer_id	name
3	Kaitlin Richards
4	Christina Williams
5	David Allen
6	Mark Duke
7	Briana Wright
8	John Bryan

At the bottom of the interface, there is a status bar showing "customers 49" and an "Apply" button.

2. List all customers who have placed an order as well as reviewed a product [intersect not supported]

Select c. customer_id,c. name

from customers as c

where Exists(Select 1 from orders as o where c.customer_id = o.customer_id)

and Exists (Select 1 from product_reviews as pr where c.customer_id = pr.customer_id);

```
181 • Select c. customer_id,c. name
182   from customers as c
183   where Exists(Select 1 from orders as o where c.customer_id = o.customer_id)
184   and Exists (Select 1 from product_reviews as pr where c.customer_id = pr.customer_id);
```

Result Grid		Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
customer_id	name				
1	Thomas Owens				
2	Charles Grant				
4	Christina Williams				
6	Mark Duke				
7	Briana Wright				
9	Jason Thompson				

customers 50 x

Result Grid

Form Editor

Apply